

COLLEGE CODE: 9513

COLLEGE NAME: Jayaraj Annapackiam CSI
College of Engineering

DEPARTMENT : Computer Science and
Engineering

STUDENT NM-ID: 5D988F851663D459EED4237D4B88CD04

ROLL NO 951323104029

DATE :13.10.2025

Completed the project name as

**Interview Buddy-AI Powered Interview
Preparation Bot(A Single Page Application)**

SUMBITED BY,

NAME: LINGA PABITHA. K

MOBILE NO:7540011729

PHASE 5

ABSTRACT

The “Interview Buddy” project is an AI-powered mock interview simulator developed using React.js. It allows users to practice interview questions interactively and receive instant feedback. The system randomizes questions, tracks answers, and calculates performance metrics to help users evaluate their readiness for interviews.

OBJECTIVES

- To simulate a real interview environment.
- To evaluate user performance with instant scoring.
- To provide a clean, user-friendly interface.
- To help students prepare confidently for interviews.

TECHNOLOGIES USED

Technology	Purpose
React.js	Frontend framework
HTML, CSS	UI design
JavaScript (ES6)	Logic and state handling
useState, useEffect	React hooks for state management
Node.js	Runtime environment

SYSTEM DESIGN

Flow Diagram:

- Home → Mock Interview → Feedback

Data Flow: User → selects section → attempts questions → gets feedback.

MODULES

Home: Introduction section displaying the project purpose.

Mock Interview: Displays random MCQs and records user answers.

Feedback: Shows performance report and score percentage.

RESULTS

The app successfully displays randomized questions, provides real-time feedback, and calculates the user’s final score accurately. It is responsive and suitable for both desktop and mobile devices.

CHALLENGES & RESOLUTIONS

Challenge

Randomizing question order

Managing multiple state updates

Component rendering delay

Navigation limits

Resolution

Implemented custom shuffleArray() function.

Used React's useState effectively.

Added conditional loading text.

Disabled buttons at first and last question.

FUTURE ENHANCEMENTS

- Add AI-generated personalized feedback.
- Integrate voice-based interview simulation.
- Add question categories and difficulty levels.
- Store user scores in a database.

CONCLUSION

This project successfully demonstrates a modern React-based application for mock interview preparation. It enhances confidence, improves readiness, and provides an engaging experience for students and job seekers.

CODE:

Index.js

```
src > JS index.js > ...
1  import React from 'react';
2  import ReactDOM from 'react-dom/client';
3  import './index.css';
4  import App from './App';
5  import reportWebVitals from './reportWebVitals';
6
7  const root = ReactDOM.createRoot(document.getElementById('root'));
8  root.render(
9    <React.StrictMode>
10     <App />
11   </React.StrictMode>
12 );
13
14 reportWebVitals();
15
```

App.js:

```
import React, { useState, useEffect } from "react";
import "./App.css";

// Shuffle function for questions only
function shuffleArray(array) {
  const newArr = [...array];
  for (let i = newArr.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1));
    [newArr[i], newArr[j]] = [newArr[j], newArr[i]];
  }
  return newArr;
}

function App() {
  const [activeSection, setActiveSection] =
    useState("home");
  const [questions, setQuestions] = useState([]);
  const [currentQ, setCurrentQ] = useState(0);
  const [selectedOptions, setSelectedOptions] =
    useState([]);
  const [scores, setScores] = useState([]);

  // Original questions with options in order
  const originalQuestions = [
    {
      question: "Which of the following is a good way
to introduce yourself?",
      options: [
        "Talk only about hobbies",
        "Briefly summarize your background and
strengths",
```

```
"Explain your entire life story",
"Just say your name and stop",
],
correctIndex: 1,
},
{
  question: "What should you focus on while
answering 'Why this company?'",
  options: [
    "The company's reputation and your alignment
with its values",
    "Salary only",
    "That you just need any job",
    "That you live nearby",
  ],
  correctIndex: 0,
},
{
  question: "What is a good way to discuss your
weaknesses?",
  options: [
    "Say you have no weaknesses",
    "Mention a weakness and how you're
improving it",
    "Complain about your boss",
    "Avoid the question",
  ],
  correctIndex: 1,
},
{
  question: "How should you answer technical
questions?",
  options: [
```

```

    "By panicking",
    "By giving clear, structured explanations",
    "By ignoring the question",
    "By saying you forgot everything",
  ],
  correctIndex: 1,
},
{
  question:
    "What should you do if you don't know the
answer to a question?",
  options: [
    "Make up something unrelated",
    "Admit you don't know and explain how you'd
find the answer",
    "Stay silent",
    "Blame your college",
  ],
  correctIndex: 1,
},
];

```

```

// Shuffle questions only
useEffect(() => {
  const shuffledQs =
shuffleArray(originalQuestions);
  setQuestions(shuffledQs);
  setSelectedOptions(Array(shuffledQs.length).fill(
null));
  setScores(Array(shuffledQs.length).fill(null));
  setCurrentQ(0);
  // eslint-disable-next-line react-hooks/exhaustive-
deps

```

```
}, []);
```

```
const handleOptionSelect = (index) => {  
  if (questions.length === 0) return;  
  const updatedSelected = [...selectedOptions];  
  updatedSelected[currentQ] = index;  
  setSelectedOptions(updatedSelected);
```

```
  
  const updatedScores = [...scores];  
  updatedScores[currentQ] =  
    index === questions[currentQ].correctIndex ? 1 :  
0;  
  setScores(updatedScores);  
};
```

```
const Home = () => (  
  <div className="section">  
    <h1>🤖 Interview Buddy</h1>  
    <p className="tagline">AI Powered Interview  
Preparation Bot</p>  
    <p>  
      Practice MCQ interview questions with  
randomized order of questions!  
    </p>  
  </div>  
)
```

```
);  
  
const MockInterview = () => {  
  if (questions.length === 0) return <p>Loading  
questions...</p>;  
  const current = questions[currentQ];
```

```

return (
  <div className="section">
    <h2>  Mock Interview</h2>
    <p className="question">
      <strong>Q {currentQ + 1} :</strong>
      {current.question}
    </p>

    <div className="options">
      {current.options.map((option, i) => (
        <label key={i} className="option-item">
          <input
            type="radio"
            name={`question-${currentQ}`}
            checked={selectedOptions[currentQ] ===
i}
            onChange={() => handleOptionSelect(i)}
          />
          {option}
        </label>
      ))}
    </div>

    {scores[currentQ] !== null && (
      <p className="score-display">
        {scores[currentQ] === 1 ? "  Correct!" :
"  Wrong Answer"}
      </p>
    )}

    <div className="button-group">
      <button

```



```

        className="primary-btn"
        onClick={() => setCurrentQ((prev) => (prev
> 0 ? prev - 1 : prev)))}
        disabled={currentQ === 0}
      >
        Previous
    </button>
    <button
        className="primary-btn"
        onClick={() =>
            setCurrentQ((prev) =>
                prev < questions.length - 1 ? prev + 1 :
prev
            )
        }
        disabled={currentQ === questions.length -
1}
    >
        Next
    </button>
</div>
</div>
);
};

```

```

const Feedback = () => {
    const answered = scores.filter((s) => s !== null);
    const total = answered.reduce((a, b) => a + b, 0);
    const totalQuestions = questions.length;
    const percentage = totalQuestions
        ? ((total / totalQuestions) * 100).toFixed(1)
        : 0;

```

```

return (
  <div className="section">
    <h2>  Feedback Summary</h2>
    <p>
      Questions Attempted:
      {answered.length}/{totalQuestions}
    </p>
    <p>Correct Answers: {total}</p>
    <p>Score Percentage: {percentage}%</p>
  </div>
);
};

```

```

return (
  <div className="app-container">
    <nav className="navbar">
      <div className="logo">Interview
Buddy</div>
      <div className="nav-links">
        <button
          className={activeSection === "home" ?
"active" : ""}
          onClick={() => setActiveSection("home")}
        >
          Home
        </button>
        <button
          className={activeSection === "mock" ?
"active" : ""}
          onClick={() => setActiveSection("mock")}
        >
          Mock Interview

```

```

        </button>
        <button
            className={activeSection === "feedback" ?
"active" : ""}
            onClick={() =>
setActiveSection("feedback")}
        >
            Feedback
        </button>
    </div>
</nav>

```

```

    <div className="content">
        {activeSection === "home" && <Home />}
        {activeSection === "mock" &&
<MockInterview />}
        {activeSection === "feedback" && <Feedback
/>}
    </div>
</div>
);
}

```

```
export default App;
```

App.css:

```

body {
    font-family: "Segoe UI", Tahoma, Geneva,
Verdana, sans-serif;
    margin: 0;
    background: linear-gradient(to right, #a1c4fd,
#e2e9fb);

```

```
    min-height: 100vh;
}
```

```
.app-container {
  max-width: 700px;
  margin: 0 auto;
  padding: 20px;
}
```

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  background-color: #1e3c72;
  color: white;
  padding: 12px 20px;
  border-radius: 8px;
  box-shadow: 0 4px 8px rgba(0, 0, 0, 0.2);
}
```

```
.navbar .logo {
  font-weight: bold;
  font-size: 1.3rem;
}
```

```
.nav-links button {
  background: transparent;
  border: none;
  color: white;
  margin-left: 12px;
  cursor: pointer;
  font-weight: bold;
  font-size: 1rem;
}
```

```
    transition: transform 0.2s;
}
```

```
.nav-links button:hover {
    transform: scale(1.05);
}
```

```
.nav-links button.active {
    text-decoration: underline;
}
```

```
.section {
    background-color: #ffffffcc;
    padding: 25px;
    margin-top: 25px;
    border-radius: 12px;
    box-shadow: 0 6px 12px rgba(0, 0, 0, 0.15);
}
```

```
.question {
    font-size: 1.2rem;
    font-weight: 500;
    margin-bottom: 15px;
    color: #1e3c72;
}
```

```
.options {
    display: flex;
    flex-direction: column;
    margin-bottom: 15px;
}
```

```
.option-item {
```

```
margin: 8px 0;
cursor: pointer;
padding: 8px 12px;
border-radius: 6px;
transition: background 0.2s, transform 0.1s;
}
```

```
.option-item:hover {
  background-color: #dbeafe;
  transform: scale(1.02);
}
```

```
.option-item input {
  margin-right: 10px;
}
```

```
.button-group {
  margin-top: 20px;
}
```

```
.primary-btn {
  background-color: #1e3c72;
  color: #fff;
  border: none;
  padding: 10px 18px;
  margin-right: 12px;
  border-radius: 6px;
  cursor: pointer;
  font-weight: bold;
  transition: background 0.3s, transform 0.2s;
}
```

```
.primary-btn:hover {
```

```
background-color: #4a69bd;
transform: scale(1.05);
}
```

```
.primary-btn:disabled {
  background-color: #b0b0b0;
  cursor: not-allowed;
}
```

```
.score-display {
  font-weight: bold;
  margin-top: 12px;
  font-size: 1.05rem;
  color: #ff3f3f;
}
```

```
.score-display.correct {
  color: #27ae60;
}
```

```
.tagline {
  font-style: italic;
  margin-bottom: 15px;
  color: #1e3c72;
}
```

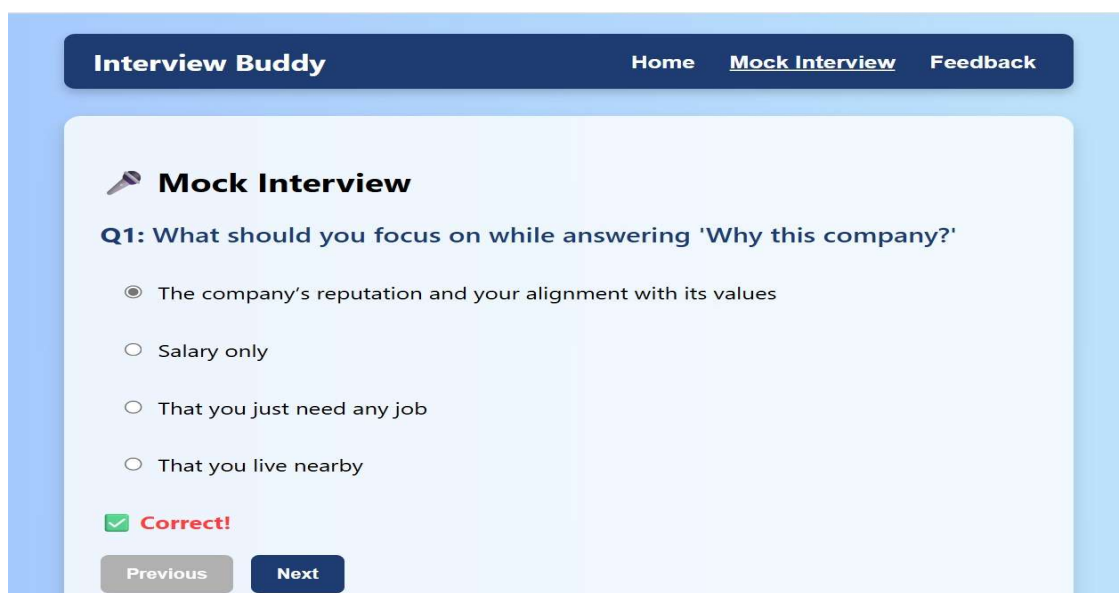
Package.json:

```
{
  "name": "react-quiz",
  "version": "0.1.0",
  "private": true,
  "dependencies": {
```

```
"@testing-library/dom": "^10.4.1",
"@testing-library/jest-dom": "^6.9.1",
"@testing-library/react": "^16.3.0",
"@testing-library/user-event": "^13.5.0",
"react": "^19.2.0",
"react-dom": "^19.2.0",
"react-scripts": "5.0.1",
"web-vitals": "^2.1.4"
},
"scripts": {
  "start": "react-scripts start",
  "build": "react-scripts build",
  "test": "react-scripts test",
  "eject": "react-scripts eject"
},
"eslintConfig": {
  "extends": [
    "react-app",
    "react-app/jest"
  ]
},
"browserslist": {
  "production": [
    ">0.2%",
    "not dead",
    "not op_mini all"
  ],
  "development": [
    "last 1 chrome version",
    "last 1 firefox version",
    "last 1 safari version"
  ]
}
```


}

Screenshot:



 Mock Interview

Q2: What should you do if you don't know the answer to a question?

- ☐ Make up something unrelated
- ☒ Admit you don't know and explain how you'd find the answer
- ☐ Stay silent
- ☐ Blame your college

 **Correct!**

[Previous](#)[Next](#) **Mock Interview**

Q3: What should you do if you don't know the answer to a question?

- ☐ Make up something unrelated
- ☐ Admit you don't know and explain how you'd find the answer
- ☐ Stay silent
- ☒ Blame your college

 **Wrong Answer**

[Previous](#)[Next](#)

 Mock Interview

Q4: What is a good way to discuss your weaknesses?

- ☐ Say you have no weaknesses
- ☒ Mention a weakness and how you're improving it
- ☐ Complain about your boss
- ☐ Avoid the question

 **Correct!**

[Previous](#)[Next](#) **Mock Interview**

Q5: Which of the following is a good way to introduce yourself?

- ☐ Talk only about hobbies
- ☒ Briefly summarize your background and strengths
- ☐ Explain your entire life story
- ☐ Just say your name and stop

 **Correct!**

[Previous](#)[Next](#)

Interview Buddy[Home](#)[Mock Interview](#)[Feedback](#)

**Feedback Summary**

Questions Attempted: 5/5

Correct Answers: 4

Score Percentage: 80.0%

LINK:

<https://github.com/klingapabitha2006-hue/Interview-Buddy-Bot.git>